

Playwright Interview Questions & Answers

Feature

Primitive Data Types

Non-Primitive Data Types

Store

Actual value

Reference to value

Mutable

Mostly immutable

Mutable

Comparison

Compared by value

Compared by reference

Memory

Stack

Heap (reference stored in stack)

Examples

string, number, boolean, null, undefined, bigint, symbol

object, array, function, class, Date, Map, Set

1. What is Playwright?

Answer:

Playwright is a Microsoft open-source automation framework for testing web applications. It supports Chromium, Firefox, and WebKit browsers and enables reliable end-to-end testing.

2. What are the key features of Playwright?

Answer:

Auto-waiting

Cross-browser support

Multi-tab and multi-context testing

Network interception

Mobile emulation

Screenshots and videos

Parallel execution

3. What languages does Playwright support?

Answer:

JavaScript, TypeScript, Python, .NET, and Java.

4. How is Playwright different from Selenium?

Answer:

Built-in auto-waiting

Faster execution

Reliable selectors

Browser contexts

Better handling of modern web apps

5. What are Browser Contexts in Playwright?

Answer:

Browser contexts are isolated browser sessions within a single browser instance.

6. What is a Locator in Playwright?

Answer:

A locator is a way to find elements in the page using user-friendly and reliable strategies.

7. How is auto-waiting implemented in Playwright?

Answer:

Playwright automatically waits for elements to become actionable before performing actions.

8. What is the role of Expect in Playwright?

Answer:

Expect is used for assertions in Playwright tests to verify application behavior.

9. What is the difference between Page and Context?

Answer:

Page: Represents a single browser tab.

Context: Represents a browser session.

10. How do you handle a browser in Playwright?

Answer:

```
const browser = await chromium.launch();
```

11. What is the use of headless mode?

Answer:

Runs the browser without a visible UI, making tests faster and suitable for CI environments.

12. How do you capture screenshots in Playwright?

Answer:

```
await page.screenshot({ path: 'image.png' });
```

13. How do you record video in Playwright?

Answer:

Enable video recording while creating a browser context.

14. How do you handle dialogs (alerts)?

Answer:

```
page.on('dialog', dialog => dialog.accept());
```

15. How do you upload files in Playwright?

Answer:

```
await page.setInputFiles('input', 'file.txt');
```

16. How do you handle popups in Playwright?

Answer:

```
const popup = await page.waitForEvent('popup');
```

17. How do you handle frames in Playwright?

Answer:

Use `frameLocator()` or `contentFrame()`.

18. How do you work with multiple tabs/windows?

Answer:

Listen for the page event and switch to the new page object.

19. How do you configure test retries?

Answer:

```
retries: 2
```

in Playwright configuration.

20. How do you capture API responses in Playwright?

Answer:

```
page.on('response', response => ...)
```

21. How do you mock API responses?

Answer:

Use `page.route()` to intercept and mock requests.

22. How do you handle authentication in Playwright?

Answer:

Use `storageState` to save and reuse authenticated sessions.

23. How do you integrate Playwright with CI/CD?

Answer:

Install dependencies and run tests using pipeline tools like Jenkins, GitHub Actions, Azure DevOps, etc.

24. How do you debug tests?

Answer:

`PWDEBUG=1`

and use Playwright Inspector.

25. How do you generate test reports?

Answer:

```
npx playwright show-report
```

26. What is Page Object Model (POM) in Playwright?

Answer:

POM is a design pattern that creates reusable and maintainable page classes for test code.

27. How do you wait in Playwright?

Answer:

Prefer:

```
locator.waitFor()
```

or

```
page.waitForSelector()
```

instead of hard waits.

28. How do you handle file uploads?

Answer:

```
locator.setInputFiles()
```

29. How do you handle scrolling in Playwright?

Answer:

```
locator.scrollIntoViewIfNeeded()
```

30. How do you handle alerts in Playwright?

Answer:

```
dialog.accept()
```

```
dialog.dismiss()
```

31. How do you perform drag and drop?

Answer:

```
source.dragTo(target)
```

32. How do you handle network failures?

Answer:

Use:

```
page.route()  
to abort or fulfill network requests.
```

33. How do you test on mobile devices?

Answer:

Use device descriptors from:

```
playwright.devices  
and launch contexts.
```

34. How do you test login functionality?

Answer:

Fill the form, click submit, and assert successful navigation or element visibility.

35. How do you handle cookies in Playwright?

Answer:

```
context.addCookies()  
context.cookies()
```

36. What is Trace Viewer in Playwright?

Answer:

Trace Viewer helps debug tests by showing step-by-step execution details.

37. How do you generate trace files?

Answer:

```
context.tracing.start()  
context.tracing.stop()
```

38. How do you perform visual testing?

Answer:

```
expect(page).toHaveScreenshot()
```

39. How do you test file downloads?

Answer:

```
page.waitForEvent('download')
```

40. How do you test WebSockets?

Answer:

Use Playwright's network monitoring capabilities to inspect WebSocket traffic.

41. How do you handle slow networks?

Answer:

Use:

```
context.setOffline(false)  
and network throttling techniques.
```

42. How do you test accessibility?

Answer:

Use:

```
@axe-core/playwright  
to validate accessibility issues.
```

43. How do you handle sessions and storage?

Answer:

Use:

`context.storageState()`
to save and restore sessions.

44. How do you test in different browsers?

Answer:

Run tests on Chromium, Firefox, and WebKit using different projects in configuration.

45. How do you handle dynamic content?

Answer:

Rely on Playwright's auto-waiting and robust locators.

46. How do you structure a Playwright project?

Answer:

Use Page Object Model, helpers, fixtures, tests, and configuration files.

47. How do you test performance?

Answer:

Use tracing, network logs, and Lighthouse for performance analysis.

48. How do you handle flaky tests?

Answer:

Use retries, stable locators, and avoid hard waits.

49. How do you test on different environments?

Answer:

Use environment variables and configuration files.

50. How do you ensure maintainability?

Answer:

Follow Page Object Model
Use reusable components
Use stable locators
Apply proper coding standards
Keep tests modular and clean

Playwright Best Practices

Use auto-waiting and reliable locators.
Prefer role and text locators.
Follow Page Object Model.
Keep tests independent and isolated.
Use fixtures for setup and teardown.
Run tests in parallel.
Capture traces/screenshots/videos for debugging.
Use configuration files for maintainability.
Keep test data separate and clean.
Follow CI/CD and DevOps best practices.
Regularly review and refactor tests.

46. How do you handle multiple selectors in Playwright?

Answer:

Use locators with conditions or custom logic until one matches.
`locator.or()`
or custom locator strategies.

47. How do you work with Shadow DOM?

Answer:

Use Playwright's built-in locator support for shadow roots.

`locator()`

can pierce shadow DOM automatically.

48. How do you handle Shadow DOM in Playwright?

Answer:

Use locators such as:

`locator()`

to interact with elements inside the shadow DOM.

49. How do you handle dynamic content?

Answer:

Prefer auto-waiting and use:

`locator.waitFor()`

to ensure elements are available.

50. What is the difference between `locator()` and `elementHandle()`?

Answer:

Locator: Recommended, auto-waiting, resilient.

ElementHandle: Reference to a specific DOM element at a point in time.

51. How do you handle stale element references?

Answer:

Use locators instead of ElementHandles because locators automatically re-resolve elements when needed.

52. What is the difference between `page.waitForSelector()` and

`locator.waitFor()`?

Answer:

`locator.waitFor()` is recommended and locator-centric.

`page.waitForSelector()` is older but still supported.

53. How do you handle file uploads in Playwright?

Answer:

`page.setInputFiles()`

54. How do you handle downloads in Playwright?

Answer:

`page.waitForEvent('download')`

and

`download.saveAs(path)`

55. How do you handle drag and drop in Playwright?

Answer:

`page.dragAndDrop()`

or

`target.dragTo()`

56. How do you handle hover in Playwright?

Answer:

`locator.hover()`

57. How do you handle keyboard actions?

Answer:

Use:

`page.keyboard`

for key presses such as Enter, Arrow keys, and shortcuts.

58. How do you switch between tabs/pages?

Answer:

`context.newPage()`

to open a tab and interact with it.

59. How do you close a browser, context, or page?

Answer:

`page.close()`

`context.close()`

`browser.close()`

60. How do you check if an element is enabled or disabled?

Answer:

`locator.isEnabled()`

`locator.isDisabled()`

61. How do you handle iFrames in Playwright?

Answer:

Use:

`frameLocator()`

or

`contentFrame()`

to interact with frame content.

62. How do you handle multiple browsers?

Answer:

Use separate browser instances or Playwright projects for Chromium, Firefox, and WebKit.

63. What is `test.only()` in Playwright?

Answer:

Runs only the specified test and skips all others.

64. What is `test.describe()` in Playwright?

Answer:

Groups related tests together.

65. What is `beforeAll()`, `afterAll()`, `beforeEach()`, and `afterEach()`?

Answer:

These are lifecycle hooks used for setup and teardown before/after tests.

66. How do you pass parameters between tests?

Answer:

Use fixtures, test context, or global setup when appropriate.

67. How do you use environment variables in Playwright?

Answer:

Use:

`process.env`

or environment variable libraries such as `dotenv`.

68. What is global setup in Playwright?

Answer:

Global setup runs once before all tests and can perform authentication, configuration, or data setup.

69. What is global teardown in Playwright?

Answer:

Global teardown runs once after all tests and performs cleanup tasks.

70. How do you run tests in headless mode?

Answer:

Use:

headless: true

in Playwright configuration.

71. How do you run tests in debug mode?

Answer:

PWDEBUG=1

or

--debug

72. How do you generate HTML reports?

Answer:

npx playwright show-report

after running tests.

73. How do you generate JSON reports?

Answer:

Configure the reporter in:

playwright.config.ts

with a JSON reporter.

74. How do you take screenshots in Playwright?

Answer:

page.screenshot()

or

locator.screenshot()

75. How do you record video in Playwright?

Answer:

Enable:

recordVideo

when creating the browser context.

76. How do you run tests in parallel?

Answer:

Playwright runs tests in parallel by default. Configure workers to control parallelism.

77. How do you run tests on multiple browsers?

Answer:

Configure projects in:

playwright.config.ts

for Chromium, Firefox, and WebKit.

78. How do you run tests on mobile devices?

Answer:

Use Playwright device descriptors:

devices['Pixel 7']

devices['iPhone 14 Pro']

79. How do you handle geolocation in Playwright?

Answer:

Set:

```
context.setGeolocation({ latitude, longitude })
```

80. How do you handle permissions in Playwright?

Answer:

```
context.grantPermissions(['camera', 'microphone', 'notifications'])
```

81. How do you handle HTTP authentication?

Answer:

Use:

```
httpCredentials
```

to configure username and password.

82. How do you mock API responses?

Answer:

```
page.route()
```

and

```
route.fulfill()
```

to mock responses.

83. How do you wait for API responses?

Answer:

```
page.waitForResponse()
```

84. How do you verify page titles and URLs?

Answer:

```
expect(page).toHaveTitle()
```

```
expect(page).toHaveURL()
```

85. How do you handle browser popups?

Answer:

Use:

```
page.waitForEvent('popup')
```

to handle newly opened windows.

86. How do you handle cookies in Playwright?

Answer:

```
context.addCookies()
```

```
context.clearCookies()
```

87. How do you store and reuse authentication state?

Answer:

Use:

```
storageState
```

to save and restore login state.

88. What is playwright.config.ts?

Answer:

It is the main configuration file used to define projects, retries, reporters, timeouts, and other settings.

89. How do you retry failed tests?

Answer:

Set:

```
retries
```

in playwright.config.ts or per test configuration.

90. How do you tag tests in Playwright?

Answer:

Use:

`test.info().annotations`

or title tags to categorize and filter tests.

Part 2 Best Practices

Use auto-waiting and locators instead of manual waits.

Prefer meaningful locators and avoid XPath when possible.

Follow the Page Object Model (POM).

Keep tests independent and isolated.

Use fixtures for setup and teardown.

Execute tests in parallel for faster feedback.

Capture traces, screenshots, and videos for debugging.

Use configuration files for environment management.

Handle exceptions gracefully.

Keep test data separate from test logic.

Follow DRY (Don't Repeat Yourself) and KISS (Keep It Simple, Stupid) principles.

Maintain clean and consistent code.

Quick Tips

Always prefer auto-waiting.

Use role-based and test-id locators.

Follow POM design patterns.

Keep frameworks modular and reusable.

Log important details for easier debugging.

Run tests in parallel whenever possible.